

Математическое моделирование

Лабораторная работа № 3

Ванчинга Дэвид

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Теоретические сведения	8
3.2	Задача	14
4	Две системы ОДУ: численное решение и визуализация	16
4.1	Инициализация проекта и загрузка пакетов	16
4.2	Начальные условия и временной интервал	17
4.3	Параметры первой системы	17
4.4	Параметры второй системы	17
4.5	Внешние воздействия для первой системы	18
4.6	Внешние воздействия для второй системы	18
4.7	Определение моделей	18
4.8	Утилита: один прогон (решение, график, таблица, сохранение) . .	19
4.9	Запуск 1: первая система	20
4.10	Запуск 2: вторая система	20
5	Параметрическое исследование двух систем ОДУ	22
5.1	Активация проекта и загрузка пакетов	22
5.2	Внешние воздействия для первой системы	23
5.3	Внешние воздействия для второй системы	23
5.4	Определение моделей	23
5.5	Определение базовых параметров в Dict	24
5.6	Функция-обертка для запуска одного эксперимента	25
5.7	Запуск базового эксперимента (линейная система)	27
5.8	Визуализация базового эксперимента	28
5.9	Второй базовый эксперимент (нелинейная система)	28
5.10	Параметрическое сканирование	30
5.11	Запуск всех экспериментов и сбор результатов	31
5.12	Анализ и визуализация результатов сканирования	33
5.13	Бенчмаркинг	37
5.14	Сохранение всех результатов	39
5.15	Базовые эксперименты	41

5.16 Параметрическое сканирование	43
5.17 Время вычислений	45
5.18 Анализ итоговой метрики norm_final	46
6 Выводы	48
Список литературы	50

Список иллюстраций

3.1 Жесткая модель войны	11
3.2 Фазовые траектории для второго случая	13
5.1 Базовый эксперимент (linear)	41
5.2 Базовый эксперимент (nonlinear)	42
5.3 Сканирование траекторий $x(t)$	43
5.4 Сканирование траекторий $y(t)$	44
5.5 Зависимость времени вычисления	45
5.6 Зависимость norm_final	46

Список таблиц

1 Цель работы

Рассмотрим некоторые простейшие модели боевых действий – модели Ланчестера. В противоборстве могут принимать участие, как регулярные войска, так и партизанские отряды. В общем случае главной характеристикой соперников являются численности сторон. Если в какой-то момент времени одна из численностей обращается в нуль, то данная сторона считается проигравшей (при условии, что численность другой стороны в данный момент положительна).

2 Задание

1. Изучить три случая модели Ланчестера
2. Построить графики изменения численности войск
3. Определить победившую сторону

3 Выполнение лабораторной работы

3.1 Теоретические сведения

Рассмотри три случая ведения боевых действий: 1. Боевые действия между регулярными войсками 2. Боевые действия с участием регулярных войск и партизанских отрядов 3. Боевые действия между партизанскими отрядами

В первом случае численность регулярных войск определяется тремя факторами:

1. скорость уменьшения численности войск из-за причин, не связанных с боевыми действиями (болезни, травмы, дезертирство);
2. скорость потерь, обусловленных боевыми действиями противоборствующих сторон (что связано с качеством стратегии, уровнем вооружения, профессионализмом солдат и т.п.);
3. скорость поступления подкрепления (задаётся некоторой функцией от времени).

В этом случае модель боевых действий между регулярными войсками описывается следующим образом

$$\begin{cases} \frac{dx}{dt} = -a(t)x(t) - b(t)y(t) + P(t) \\ \frac{dy}{dt} = -c(t)x(t) - h(t)y(t) + Q(t) \end{cases}$$

Потери, не связанные с боевыми действиями, описывают члены $-a(t)x(t)$

и $-h(t)y(t)$, члены $-b(t)y(t)$ и $-c(t)x(t)$ отражают потери на поле боя. Коэффициенты $b(t)$, $c(t)$ указывают на эффективность боевых действий со стороны y и x соответственно, $a(t), h(t)$ - величины, характеризующие степень влияния различных факторов на потери. Функции $P(t), Q(t)$ учитывают возможность подхода подкрепления к войскам X и Y в течение одного дня.

Во втором случае в борьбу добавляются партизанские отряды. Нерегулярные войска в отличии от постоянной армии менее уязвимы, так как действуют скрытно, в этом случае сопернику приходится действовать неизбирательно, по площадям, занимаемым партизанами. Поэтому считается, что темп потерь партизан, проводящих свои операции в разных местах на некоторой известной территории, пропорционален не только численности армейских соединений, но и численности самих партизан. В результате модель принимает вид:

$$\begin{cases} \frac{dx}{dt} = -a(t)x(t) - b(t)y(t) + P(t) \\ \frac{dy}{dt} = -c(t)x(t)y(t) - h(t)y(t) + Q(t) \end{cases}$$

Модель ведение боевых действий между партизанскими отрядами с учетом предположений, сделанном в предыдущем случае, имеет вид:

$$\begin{cases} \frac{dx}{dt} = -a(t)x(t) - b(t)x(t)y(t) + P(t) \\ \frac{dy}{dt} = -h(t)y(t) - c(t)x(t)y(t) + Q(t) \end{cases}$$

В простейшей модели борьбы двух противников коэффициенты $b(t)$ и $c(t)$ являются постоянными. Попросту говоря, предполагается, что каждый солдат армии x убивает за единицу времени c солдат армии y (и, соответственно, каждый солдат армии y убивает b солдат армии x). Также не учитываются потери, не связанные с боевыми действиями, и возможность подхода подкрепления. Состояние системы описывается точкой (x, y) положительного квадранта плоскости. Координаты этой точки, x и y - это численности противостоящих армий. Тогда модель принимает вид

$$\begin{cases} \frac{dx}{dt} = -by \\ \frac{dy}{dt} = -ax \end{cases}$$

Это - жесткая модель, которая допускает точное решение

$$\frac{dx}{dy} = \frac{by}{cx}$$

$$cxdx = bydy, cx^2 - by^2 = C$$

Эволюция численностей армий x и y происходит вдоль гиперболы, заданной этим уравнением (рис. [fig:001]). По какой именно гиперболе пойдет война, зависит от начальной точки.

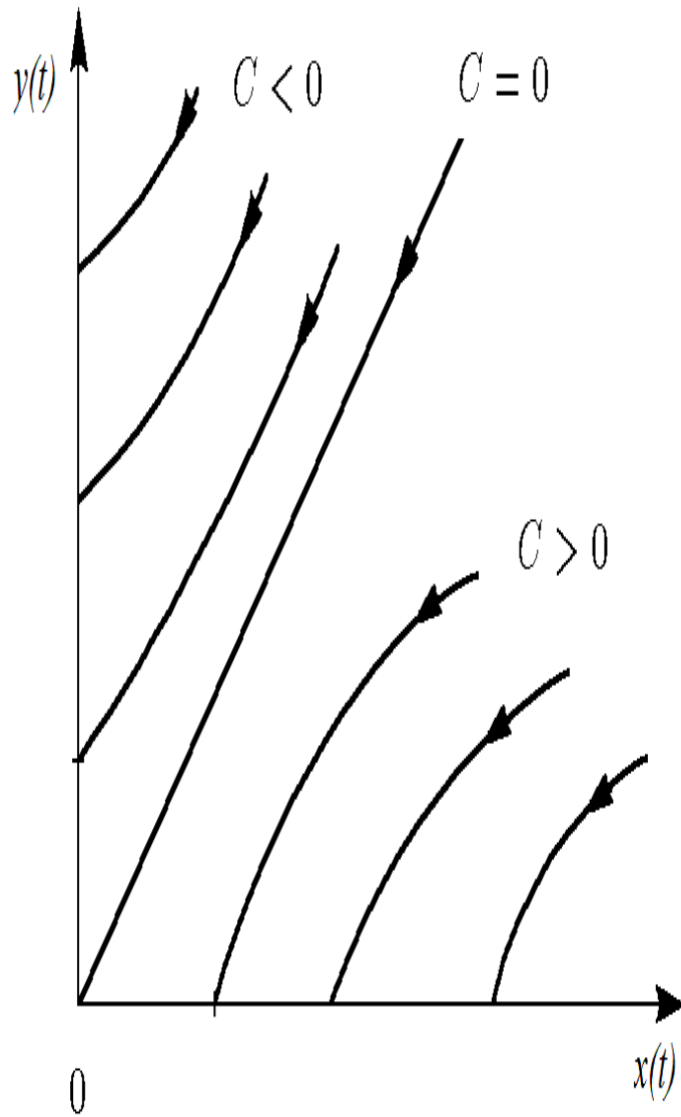


Рисунок 3.1: Жесткая модель войны

Эти гиперболы разделены прямой $\sqrt{cx} = \sqrt{by}$. Если начальная точка лежит выше этой прямой, то гипербола выходит на ось y . Это значит, что в ходе войны численность армии x уменьшается до нуля (за конечное время). Армия y выигрывает, противник уничтожен. Если начальная точка лежит ниже, то выигрывает армия x . В разделяющем эти случаи состоянии (на прямой) война заканчивается истреблением обеих армий. Но на это требуется бес-

конечно большое время: конфликт продолжает тлеть, когда оба противника уже обессилены. Вывод модели таков: для борьбы с вдвое более многочисленным противником нужно в четыре раза более мощное оружие, с втрое более многочисленным - в девять раз и т. д. (на это указывают квадратные корни в уравнении прямой). Стоит помнить, что эта модель сильно идеализирована и неприменима к реальной ситуации. Но может использоваться для начального анализа. Если рассматривать второй случай (война между регулярными войсками и партизанскими отрядами) с теми же упрощениями, то модель принимает вид:

$$\begin{cases} \frac{dx}{dt} = -by(t) \\ \frac{dy}{dt} = -cx(t)y(t) \end{cases}$$

Эта система приводится к уравнению $\frac{d}{dt} (\frac{b}{2}x^2(t) - cy(t)) = 0$ которое при заданных начальных условиях имеет единственное решение: $\frac{b}{2}x^2(t) - cy(t) = \frac{b}{2}x^2(0) - cy(0) = C_1$

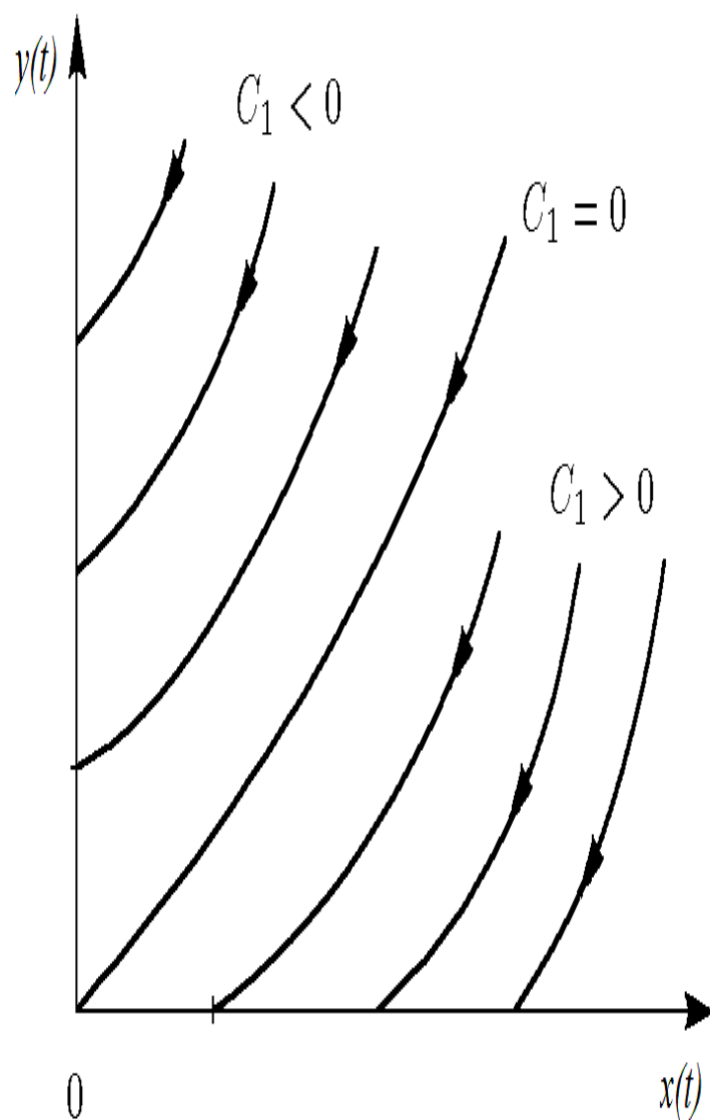


Рисунок 3.2: Фазовые траектории для второго случая

Из Рисунка **fig:002** видно, что при $C_1 > 0$ побеждает регулярная армия, при $C_1 < 0$ побеждают партизаны. Аналогично противостоянию регулярных войск, победа обеспечивается не только начальной численностью, но и боевой выручкой и качеством вооружения. При $C_1 > 0$ получаем соотношение $\frac{b}{2}x^2(0) > cy(0)$. Чтобы одержать победу партизанам необходимо увеличить коэффициент c и повысить свою начальную численность на соответствующую величину. При-

чем это увеличение, с ростом начальной численности регулярных войск $x(0)$ должно расти не линейно, а пропорционально второй степени $x(0)$. Таким образом, можно сделать вывод, что регулярные войска находятся в более выгодном положении, так как неравенство для них выполняется при меньшем росте начальной численности войск. Рассмотренные простейшие модели соперничества соответствуют системам обыкновенных дифференциальных уравнений второго порядка, широко распространенным при описании многих естественно научных объектов.

3.2 Задача

Между страной X и страной Y идет война. Численность состава войск исчисляется от начала войны, и являются временными функциями $x(t)$ и $y(t)$. В начальный момент времени страна X имеет армию численностью 32888 человек, а в распоряжении страны Y армия численностью в 17777 человек. Для упрощения модели считаем, что коэффициенты a, b, c, h постоянны. Также считаем $P(t), Q(t)$ непрерывные функции. Постройте графики изменения численности войск армии X и армии Y для следующих случаев:

1. Модель боевых действий между регулярными войсками

$$\begin{cases} \frac{dx}{dt} = -0.55x(t) - 0.77y(t) + 1.5 * \sin(3t + 1) \\ \frac{dy}{dt} = -0.66x(t) - 0.44y(t) + 1.2 * \cos(t + 1) \end{cases}$$

2. Модель ведение боевых действий с участием регулярных войск и партизанских отрядов

$$\begin{cases} \frac{dx}{dt} = -0.27x(t) - 0.88y(t) + \sin(20t) \\ \frac{dy}{dt} = -0.68x(t)y(t) - 0.37y(t) + \cos(10t) \end{cases}$$

Для моделирования процесса и построения графиков использовались внешние файлы с программным кодом:

4 Две системы ОДУ: численное решение и визуализация

Цель: решить две системы ОДУ, построить графики решений, сохранить изображения и таблицы с результатами.

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```


4.2 Начальные условия и временной интервал

```
x0 = 32888.0  
y0 = 17777.0  
t0 = 0.0  
tmax = 1.0  
  
u0 = [x0, y0]  
tspan = (t0, tmax)
```

4.3 Параметры первой системы

```
a = 0.55  
b = 0.77  
c = 0.66  
d = 0.44  
  
p1 = (a = a, b = b, c = c, d = d)
```

4.4 Параметры второй системы

```
a2 = 0.27  
b2 = 0.88  
c2 = 0.68  
d2 = 0.37  
  
p2 = (a = a2, b = b2, c = c2, d = d2)
```

4.5 Внешние воздействия для первой системы

```
function P(t)
    return 1.5 * sin(3 * t + 1)
end

function Q(t)
    return 1.2 * cos(t + 1)
end
```

4.6 Внешние воздействия для второй системы

```
function P2(t)
    return sin(20 * t)
end

function Q2(t)
    return cos(10 * t) + 1
end
```

4.7 Определение моделей

Первая система: $dx/dt = -ax - by + P(t)$ $dy/dt = -cx - dy + Q(t)$

```
function syst1!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P(t)
    dy[2] = -p.c * y[1] - p.d * y[2] + Q(t)
end
```

Вторая система: $dx/dt = -ax - by + P2(t)$ $dy/dt = -cxy - d*y + Q2(t)$

```
function syst2!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P2(t)
    dy[2] = -p.c * y[1] * y[2] - p.d * y[2] + Q2(t)
end
```

4.8 Утилита: один прогон (решение, график, таблица, сохранение)

```
function run_case(case_name; model!, u0, tspan, p, nt=100)
```

— Сетка по времени —

```
tgrid = collect(LinRange(tspan[1], tspan[2], nt))
```

— Решение ОДУ —

```
prob = ODEProblem(model!, u0, tspan, p)
sol = solve(prob, Tsit5(), saveat=tgrid)
```

— Таблица с результатами —

```
df = DataFrame(
    t = sol.t,
    x = first(sol.u),
    y = last(sol.u)
)
```

— Визуализация —

```

plt = plot(sol,
            xlabel = "t",
            ylabel = "Значение",
            label = ["x(t)" "y(t)"],
            lw = 2,
            title = "Решение системы – $case_name",
            legend = :topright
        )

```

— Сохранение —

```

savefig(plt, plotsdir(script_name, "$case_name.png"))
@save datadir(script_name, "data_$case_name.jld2") df p u0 tspan nt

return (plt = plt, df = df, sol = sol)
end

```

4.9 Запуск 1: первая система

```

res1 = run_case("system1"; model! = syst1!, u0 = u0, tspan = tspan, p = p1, nt = 100)

println("Система 1 – первые 5 строк:")
println(first(res1.df, 5))

```

4.10 Запуск 2: вторая система

```
res2 = run_case("system2"; model! = syst2!, u0 = u0, tspan = tspan, p = p2, nt = 100)

println("\nСистема 2 – первые 5 строк:")
println(first(res2.df, 5))
```

(опционально) показать последний график в интерактивной среде

```
res2.plt
```

5 Параметрическое исследование двух систем ОДУ

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
```

Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.2 Внешние воздействия для первой системы

```
function P(t)
    return 1.5 * sin(3 * t + 1)
end

function Q(t)
    return 1.2 * cos(t + 1)
end
```

5.3 Внешние воздействия для второй системы

```
function P2(t)
    return sin(20 * t)
end

function Q2(t)
    return cos(10 * t) + 1
end
```

5.4 Определение моделей

Первая система: $dx/dt = -ax - by + P(t)$ $dy/dt = -cx - dy + Q(t)$

```
function syst_linear!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P(t)
    dy[2] = -p.c * y[1] - p.d * y[2] + Q(t)
end
```

Вторая система: $dx/dt = -ax - by + P2(t)$ $dy/dt = -cxy - d*y + Q2(t)$

```
function syst_nonlinear!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P2(t)
    dy[2] = -p.c * y[1] * y[2] - p.d * y[2] + Q2(t)
end
```

5.5 Определение базовых параметров в Dict

model_type управляет выбором системы: :linear -> первая система :nonlinear
-> вторая система

```
base_params = Dict(
    :x0 => 32888.0,
    :y0 => 17777.0,

    :tspan => (0.0, 1.0),
    :nt => 100,

    :model_type => :linear,          # :linear или :nonlinear
```

Параметры первой системы

```
:a => 0.55,
:b => 0.77,
:c => 0.66,
:d => 0.44,
```

Параметры второй системы


```

:a2 => 0.27,
:b2 => 0.88,
:c2 => 0.68,
:d2 => 0.37,

:solver => Tsit5(),
:experiment_name => "base_experiment"
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end

```

5.6 Функция-обертка для запуска одного эксперимента

```

function run_single_experiment(params::Dict)
    @unpack x0, y0, tspan, nt, model_type, a, b, c, d, a2, b2, c2, d2, solver = params

    u0 = [x0, y0]
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))

    if model_type == :linear
        p = (a=a, b=b, c=c, d=d)
        prob = ODEProblem(syst_linear!, u0, tspan, p)
    else

```

```

    p = (a=a2, b=b2, c=c2, d=d2)
    prob = ODEProblem(syst_nonlinear!, u0, tspan, p)
end

sol = solve(prob, solver; saveat=tgrid)

x_vals = first(sol.u)
y_vals = last(sol.u)

```

— Мини-анализ —

```

x_final = x_vals[end]
y_final = y_vals[end]

x_max_abs = maximum(abs.(x_vals))
y_max_abs = maximum(abs.(y_vals))

norm_final = sqrt(x_final^2 + y_final^2)

return Dict(
    "solution" => sol,
    "t_points" => sol.t,
    "x_values" => x_vals,
    "y_values" => y_vals,

    "x_final" => x_final,
    "y_final" => y_final,
    "x_max_abs" => x_max_abs,
    "y_max_abs" => y_max_abs,

```

```

        "norm_final" => norm_final,

        "parameters" => params
    )
end

```

5.7 Запуск базового эксперимента (линейная система)

```

data_base, path_base = produce_or_load(
    datadir(script_name, "single"),
    base_params,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента:")
println(" x_final: ", data_base["x_final"])
println(" y_final: ", data_base["y_final"])
println(" x_max_abs: ", data_base["x_max_abs"])
println(" y_max_abs: ", data_base["y_max_abs"])
println(" norm_final: ", round(data_base["norm_final"]; digits=4))
println(" Файл результатов: ", path_base)

```

5.8 Визуализация базового эксперимента

```
p1 = plot(
    data_base["t_points"], data_base["x_values"],
    label="x(t)",
    xlabel="t",
    ylabel="Значение",
    title="Базовый эксперимент (model_type=$(base_params[:model_type]))",
    lw=2,
    legend=:topright,
    grid=true
)
plot!(
    p1,
    data_base["t_points"], data_base["y_values"],
    label="y(t)",
    lw=2
)

savefig(p1, plotsdir(script_name, "single_experiment_linear.png"))
```

5.9 Второй базовый эксперимент (нелинейная система)

```
base_params2 = copy(base_params)
base_params2[:model_type] = :nonlinear
base_params2[:experiment_name] = "base_experiment_nonlinear"
```

```

data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),
    base_params2,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты второго базового эксперимента:")
println(" x_final: ", data_base2["x_final"])
println(" y_final: ", data_base2["y_final"])
println(" x_max_abs: ", data_base2["x_max_abs"])
println(" y_max_abs: ", data_base2["y_max_abs"])
println(" norm_final: ", round(data_base2["norm_final"]; digits=4))
println(" Файл результатов: ", path_base2)

p1b = plot(
    data_base2["t_points"], data_base2["x_values"],
    label="x(t)",
    xlabel="t",
    ylabel="Значение",
    title="Базовый эксперимент (model_type=$(base_params2[:model_type]))",
    lw=2,
    legend=:topright,
    grid=true
)
plot!(

```

```

    p1b,
    data_base2["t_points"], data_base2["y_values"],
    label="y(t)",
    lw=2
)

savefig(p1b, plotsdir(script_name, "single_experiment_nonlinear.png"))

```

5.10 Параметрическое сканирование

Сканируем параметр alpha: для linear alpha -> a для nonlinear alpha -> a2

```

param_grid = Dict(
    :x0 => [32888.0],
    :y0 => [17777.0],

    :tspan => [(0.0, 1.0)],
    :nt => [100],

    :model_type => [:linear, :nonlinear],

    :a => [0.30, 0.55, 0.80, 1.00],      # для linear
    :b => [0.77],
    :c => [0.66],
    :d => [0.44],

    :a2 => [0.15, 0.27, 0.40, 0.60],    # для nonlinear
    :b2 => [0.88],

```

```

:c2 => [0.68],
:d2 => [0.37],

:solver => [Tsit5()],
:experiment_name => ["parametric_scan"]
)

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые model_type: ", param_grid[:model_type])
println("Исследуемые a: ", param_grid[:a])
println("Исследуемые a2: ", param_grid[:a2])
println("="^60)

```

5.11 Запуск всех экспериментов и сбор результатов

```

all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println("Порядок: $i/$(length(all_params)) | model_type=$(params[:model_type]) | ")

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),

```

```

    params,
    run_single_experiment;
    prefix = "scan",
    tag = false,
    verbose = false
)

result_summary = merge(
  params,
  Dict(
    :x_final => data["x_final"],
    :y_final => data["y_final"],
    :x_max_abs => data["x_max_abs"],
    :y_max_abs => data["y_max_abs"],
    :norm_final => data["norm_final"],
    :filepath => path
  )
)
push!(all_results, result_summary)

df = DataFrame(
  t = data["t_points"],
  x = data["x_values"],
  y = data["y_values"],
  model_type = fill(string(params[:model_type]), length(data["t_points"])),
  a = fill(params[:a], length(data["t_points"])),
  a2 = fill(params[:a2], length(data["t_points"]))
)

```



```
push!(all_dfs, df)
end
```

5.12 Анализ и визуализация результатов сканирования

```
results_df = DataFrame(all_results)
println("\nСводная таблица результатов (первые строки):")
println(first(results_df, 10))
```

Сравнительный график $x(t)$ для всех комбинаций

```
p2 = plot(size=(900, 520), dpi=150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :linear ?
        "linear, a=$(params[:a])" :
        "nonlinear, a2=$(params[:a2])"

    plot!(
```

```

        p2,
        data["t_points"], data["x_values"],
        label=label_text,
        lw=2,
        alpha=0.8
    )
end
plot!(
    p2,
    xlabel="t",
    ylabel="x(t)",
    title="Сканирование: траектории x(t) для разных параметров",
    legend=:outerright,
    grid=true
)
savefig(p2, plotsdir(script_name, "parametric_scan_x_comparison.png"))

```

Сравнительный график $y(t)$ для всех комбинаций

```

p3 = plot(size=(900, 520), dpi=150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

```

```

label_text = params[:model_type] == :linear ?
    "linear, a=$(params[:a])" :
    "nonlinear, a2=$(params[:a2])"

plot!(
    p3,
    data["t_points"], data["y_values"],
    label=label_text,
    lw=2,
    alpha=0.8
)
end
plot!(
    p3,
    xlabel="t",
    ylabel="y(t)",
    title="Сканирование: траектории y(t) для разных параметров",
    legend=:outerright,
    grid=true
)
savefig(p3, plotsdir(script_name, "parametric_scan_y_comparison.png"))

```

График нормы финального состояния

```

p4 = plot(size=(900, 520), dpi=150)

linear_sub = results_df[results_df.model_type .== :linear, :]
nonlinear_sub = results_df[results_df.model_type .== :nonlinear, :]

```

```

if nrow(linear_sub) > 0
  plot!(
    p4,
    linear_sub.a, linear_sub.norm_final,
    seriestype=:scatter,
    label="linear"
  )
end

if nrow(nonlinear_sub) > 0
  plot!(
    p4,
    nonlinear_sub.a2, nonlinear_sub.norm_final,
    seriestype=:scatter,
    label="nonlinear"
  )
end

plot!(
  p4,
  xlabel="Параметр модели",
  ylabel="norm_final",
  title="Зависимость norm_final от параметра модели",
  legend=:topleft,
  grid=true
)

savefig(p4, plotsdir(script_name, "norm_final_vs_parameter.png"))

```

5.13 Бенчмаркинг

```
println("\n" * "="^60)
println("БЕНЧМАРКИНГ ДЛЯ РАЗНЫХ ПАРАМЕТРОВ")
println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        u0 = [params[:x0], params[:y0]]

        if params[:model_type] == :linear
            p = (a=params[:a], b=params[:b], c=params[:c], d=params[:d])
            prob = ODEProblem(syst_linear!, u0, params[:tspan], p)
        else
            p = (a=params[:a2], b=params[:b2], c=params[:c2], d=params[:d2])
            prob = ODEProblem(syst_nonlinear!, u0, params[:tspan], p)
        end

        return solve(
            prob,
            params[:solver];
            saveat=LinRange(params[:tspan][1], params[:tspan][2], params[:nt])
        )
    end

    println("\nБенчмарк для model_type=$(params[:model_type]), a=$(params[:a]), a2=$(params[:a2]), b=$(params[:b]), b2=$(params[:b2]), c=$(params[:c]), c2=$(params[:c2]), d=$(params[:d]), d2=$(params[:d2]), solver=$(params[:solver]), nt=$(params[:nt]), tspan=$(params[:tspan])")
    benchmark_results = push!(benchmark_results, (params, benchmark_run()))
end
```

```

b = @benchmark $benchmark_run() samples=80 evals=1
tsec = median(b).time / 1e9
println(" Медианное время: ", round(tsec; digits=6), " сек")

push!(benchmark_results, (
    model_type = string(params[:model_type]),
    a = params[:a],
    a2 = params[:a2],
    time = tsec
))
end

bench_df = DataFrame(benchmark_results)

```

График времени вычисления

```

p5 = plot(size=(900, 520), dpi=150)

bench_linear = bench_df[bench_df.model_type .== "linear", :]
bench_nonlinear = bench_df[bench_df.model_type .== "nonlinear", :]

if nrow(bench_linear) > 0
    plot!(
        p5,
        bench_linear.a, bench_linear.time,
        seriestype=:scatter,
        label="linear"
    )
end

```

```

if nrow(bench_nonlinear) > 0
    plot!(
        p5,
        bench_nonlinear.a2, bench_nonlinear.time,
        seriestype=:scatter,
        label="nonlinear"
    )
end

plot!(
    p5,
    xlabel="Параметр модели",
    ylabel="Время вычисления, сек",
    title="Зависимость времени решения ODE от параметров модели",
    legend=:topleft,
    grid=true
)

savefig(p5, plotsdir(script_name, "computation_time_vs_parameter.png"))

```

5.14 Сохранение всех результатов

```

@save datadir(script_name, "all_results.jld2") base_params base_params2 param_grid al
@save datadir(script_name, "all_plots.jld2") p1 p1b p2 p3 p4 p5

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)

```

```
println("\nРезультаты сохранены в:")
println(" • data/${script_name}/single/ - базовые эксперименты")
println(" • data/${script_name}/parametric_scan/ - параметрическое сканирование")
println(" • data/${script_name}/all_results.jld2 - сводные данные")
println(" • plots/${script_name}/ - все графики")
println(" • data/${script_name}/all_plots.jld2 - объекты графиков")
println("\nДля анализа результатов используйте:")
println(" using JLD2, DataFrames")
println(" @load \"data/${script_name}/all_results.jld2\"")
println(" println(results_df)")
```


5.15 Базовые эксперименты

5.15.1 Линейная модель (model_type = linear)

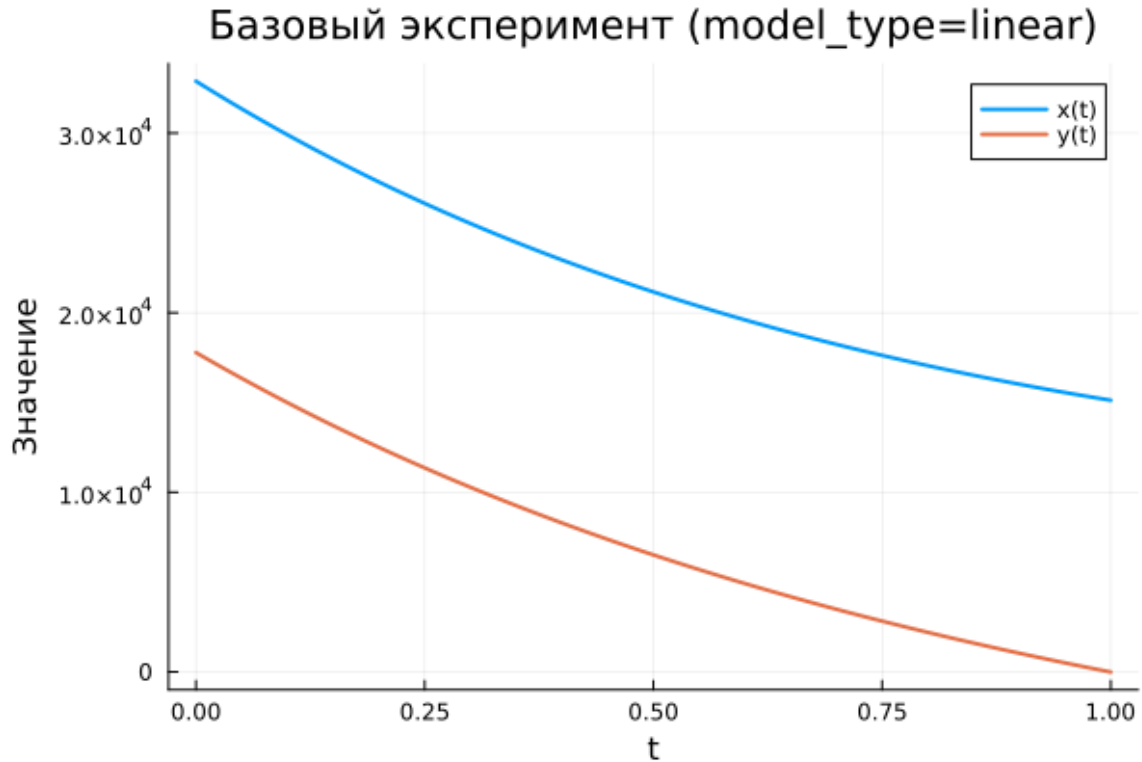


Рисунок 5.1: Базовый эксперимент (linear)

На графике показана динамика переменных $x(t)$ и $y(t)$ для линейной модели. Обе функции монотонно уменьшаются по мере увеличения времени.

Переменная $x(t)$ убывает плавно и сохраняет положительные значения на всём интервале интегрирования. Это свидетельствует о постепенном затухании состояния системы без резких переходов.

Переменная $y(t)$ убывает быстрее и к концу интервала практически достигает нуля. Такое поведение характерно для линейных систем, в которых решение экспоненциально стремится к равновесному состоянию.

Таким образом, линейная модель демонстрирует устойчивую и предсказуемую динамику: обе переменные постепенно уменьшаются без скачков и

нелинейных эффектов.

5.15.2 Нелинейная модель (model_type = nonlinear)

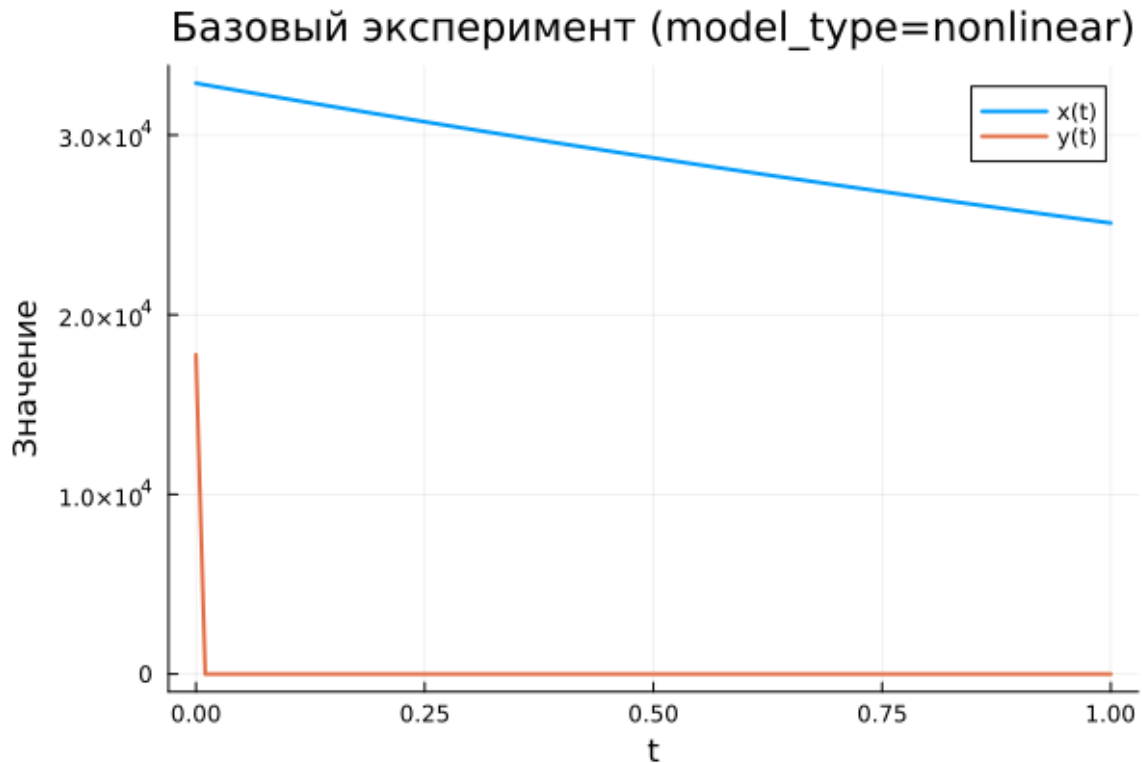


Рисунок 5.2: Базовый эксперимент (nonlinear)

Для нелинейной модели характер поведения существенно отличается. Переменная $x(t)$ убывает медленнее по сравнению с линейным случаем и остаётся на достаточно высоком уровне на всём интервале времени.

В то же время переменная $y(t)$ практически мгновенно падает до нуля уже в начальный момент времени. После этого её значение остаётся близким к нулю.

Такое поведение связано с наличием нелинейных членов в системе уравнений, которые усиливают скорость затухания одной из переменных. В результате система быстро переходит в состояние, где динамика определяется практически только переменной $x(t)$.

5.16 Параметрическое сканирование

5.16.1 Траектории $x(t)$ для различных параметров

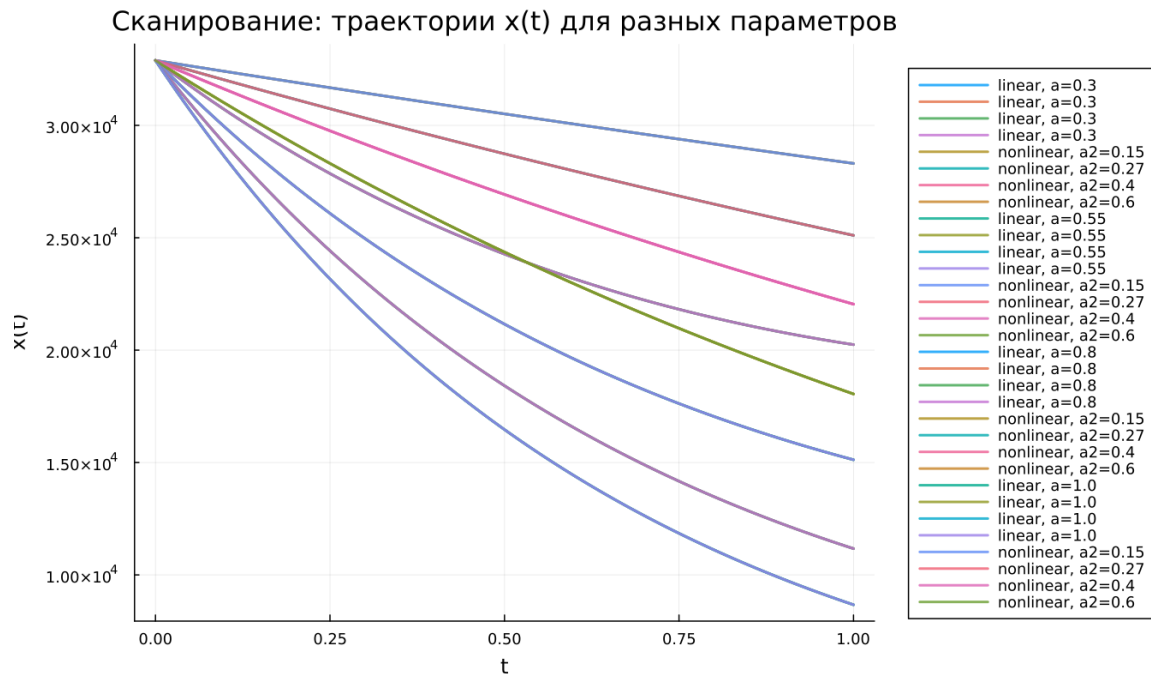


Рисунок 5.3: Сканирование траекторий $x(t)$

Было проведено сканирование параметров модели для линейного и нелинейного вариантов. Для линейной модели варьировался параметр a , а для нелинейной — параметр a_2 .

На графике видно, что при увеличении параметра скорость убывания функции $x(t)$ возрастает. Это приводит к более быстрому снижению значения переменной к концу временного интервала.

Основные наблюдения:

- при малых значениях параметра уменьшение $x(t)$ происходит медленно;
- при больших значениях параметра траектория становится более крутой;
- различия между траекториями хорошо заметны уже в середине интервала интегрирования.

Для нелинейной модели влияние параметра также сохраняется, однако характер кривых становится более сложным из-за нелинейных эффектов.

5.16.2 Траектории $y(t)$ для различных параметров

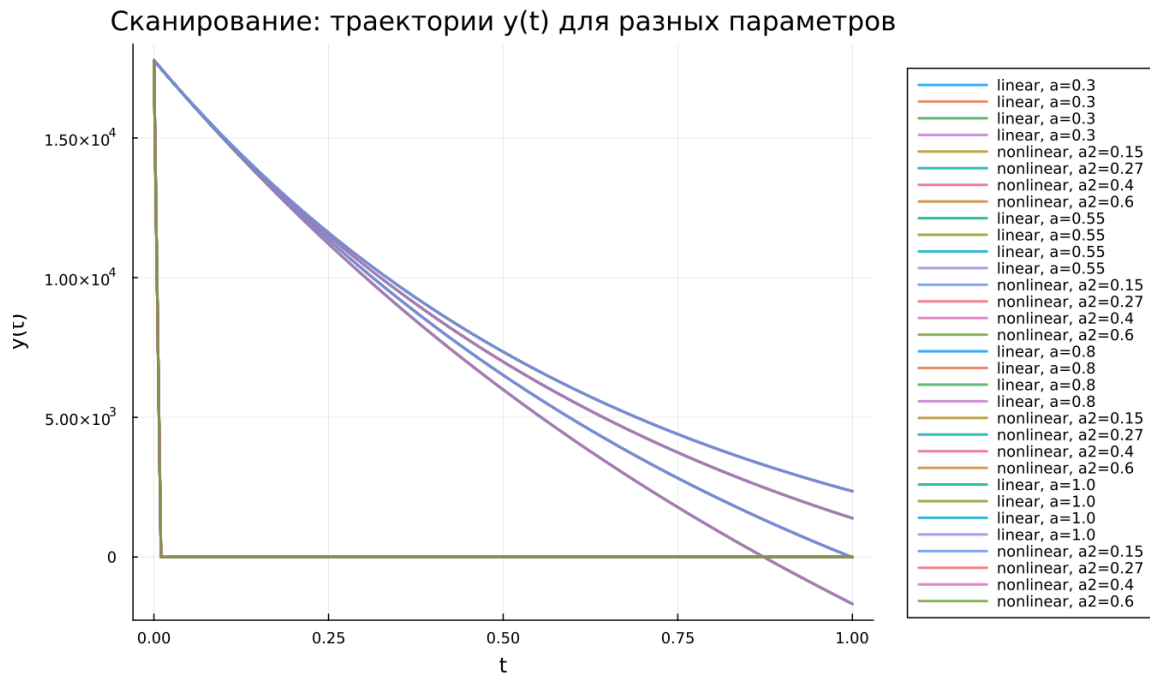


Рисунок 5.4: Сканирование траекторий $y(t)$

Аналогичное исследование было проведено для переменной $y(t)$.

В линейной модели изменение параметра приводит к постепенному изменению формы кривой. При увеличении параметра скорость убывания возрастает, и значение функции быстрее стремится к нулю.

В нелинейной модели динамика отличается: переменная $y(t)$ практически мгновенно обнуляется независимо от значения параметра. Это указывает на сильное влияние нелинейных членов, которые быстро подавляют динамику этой переменной.

Таким образом, параметр оказывает заметное влияние на линейную модель, тогда как в нелинейной системе поведение $y(t)$ остаётся почти неизменным.

5.17 Время вычислений

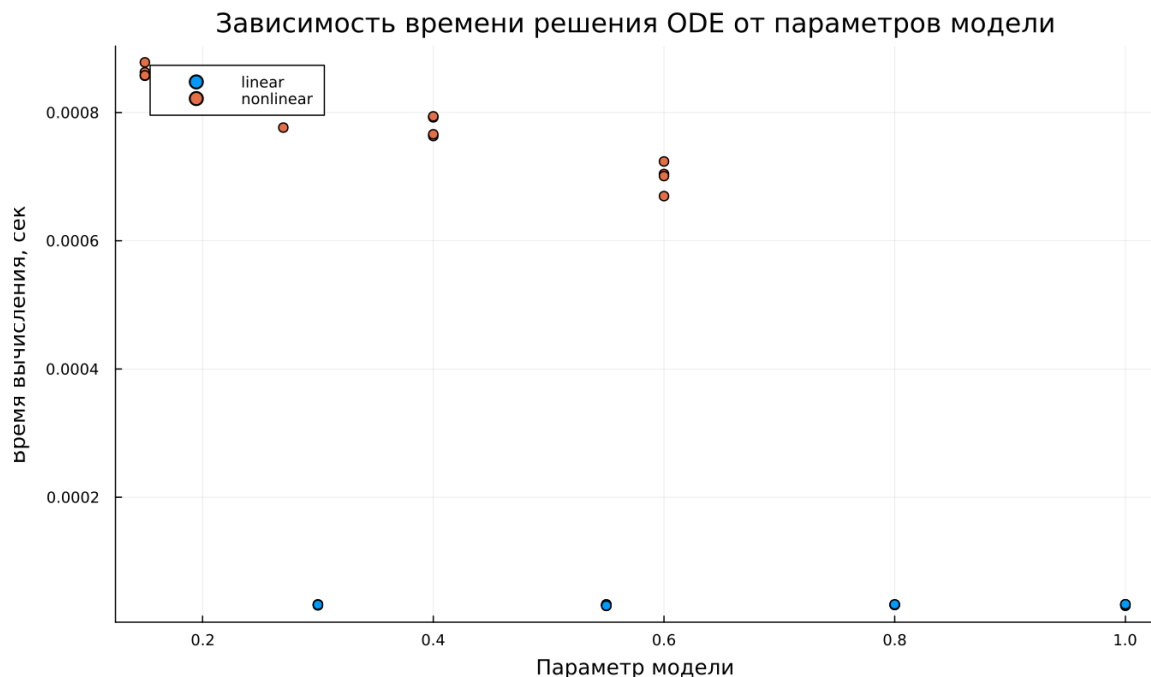


Рисунок 5.5: Зависимость времени вычисления

Был проведён бенчмаркинг численного решения системы дифференциальных уравнений при различных значениях параметров.

Из графика можно сделать следующие выводы:

- линейная модель вычисляется значительно быстрее;
- время решения линейной системы находится на уровне порядка 10^{-5} секунд;
- нелинейная модель требует больше вычислительных ресурсов — около $7 \cdot 10^{-4}$ – $9 \cdot 10^{-4}$ секунд.

Несмотря на увеличение вычислительных затрат, абсолютное время расчёта остаётся крайне малым и не представляет практических ограничений для моделирования.

5.18 Анализ итоговой метрики norm_final

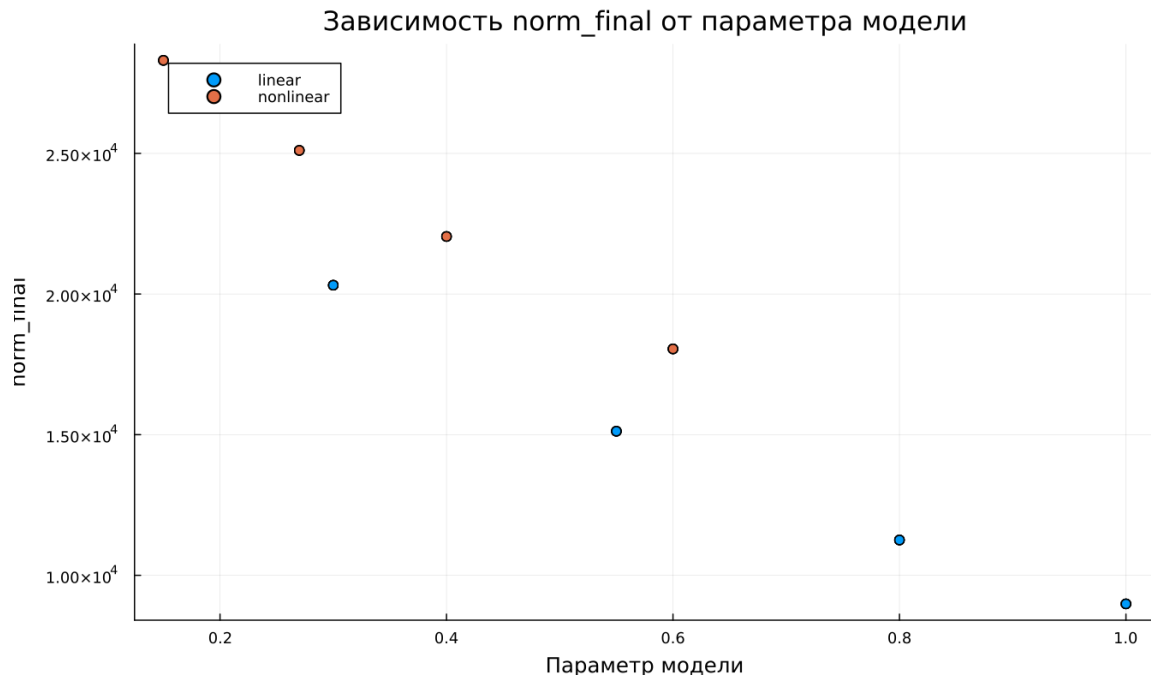


Рисунок 5.6: Зависимость norm_final

В качестве итоговой характеристики системы была рассмотрена метрика

$$\text{norm_final} = \sqrt{x(t_{final})^2 + y(t_{final})^2}$$

которая отражает величину состояния системы в конце интервала моделирования.

Анализ графика показывает:

- при увеличении параметра значение метрики уменьшается;
- для линейной модели наблюдается более быстрое снижение нормы;
- нелинейная модель сохраняет более высокие значения метрики при тех же параметрах.

Это означает, что линейная система быстрее стремится к состоянию покоя,

тогда как нелинейная модель демонстрирует более медленное затухание основной переменной.

6 Выводы

1. Линейная модель демонстрирует плавное и предсказуемое затухание обеих переменных. Функции $x(t)$ и $y(t)$ монотонно уменьшаются во времени, причём $y(t)$ стремится к нулю быстрее, чем $x(t)$.
2. В нелинейной модели наблюдается существенно иная динамика: переменная $y(t)$ практически мгновенно обнуляется, после чего система фактически описывается динамикой только переменной $x(t)$.
3. Параметры модели существенно влияют на скорость убывания решений. При увеличении параметров линейной модели траектории становятся более крутыми, что приводит к более быстрому снижению значений переменных.
4. В нелинейной системе влияние параметров на переменную $y(t)$ оказывается слабым, поскольку её значение быстро подавляется нелинейными членами системы.
5. Численный эксперимент показывает, что линейная модель вычисляется значительно быстрее, однако даже для нелинейной системы время расчёта остаётся очень малым (порядка 10^{-4} – 10^{-3} секунд).
6. Итоговая метрика состояния системы `norm_final` уменьшается при увеличении параметров модели, что указывает на усиление затухания динамики системы.

Полученные результаты согласуются с ожидаемым поведением линейных и нелинейных дифференциальных систем и подтверждают корректность численного моделирования.

Список литературы

1. Законы Осипова — Ланчестера
2. Дифференциальные уравнения динамики боя
3. Элементарные модели боя